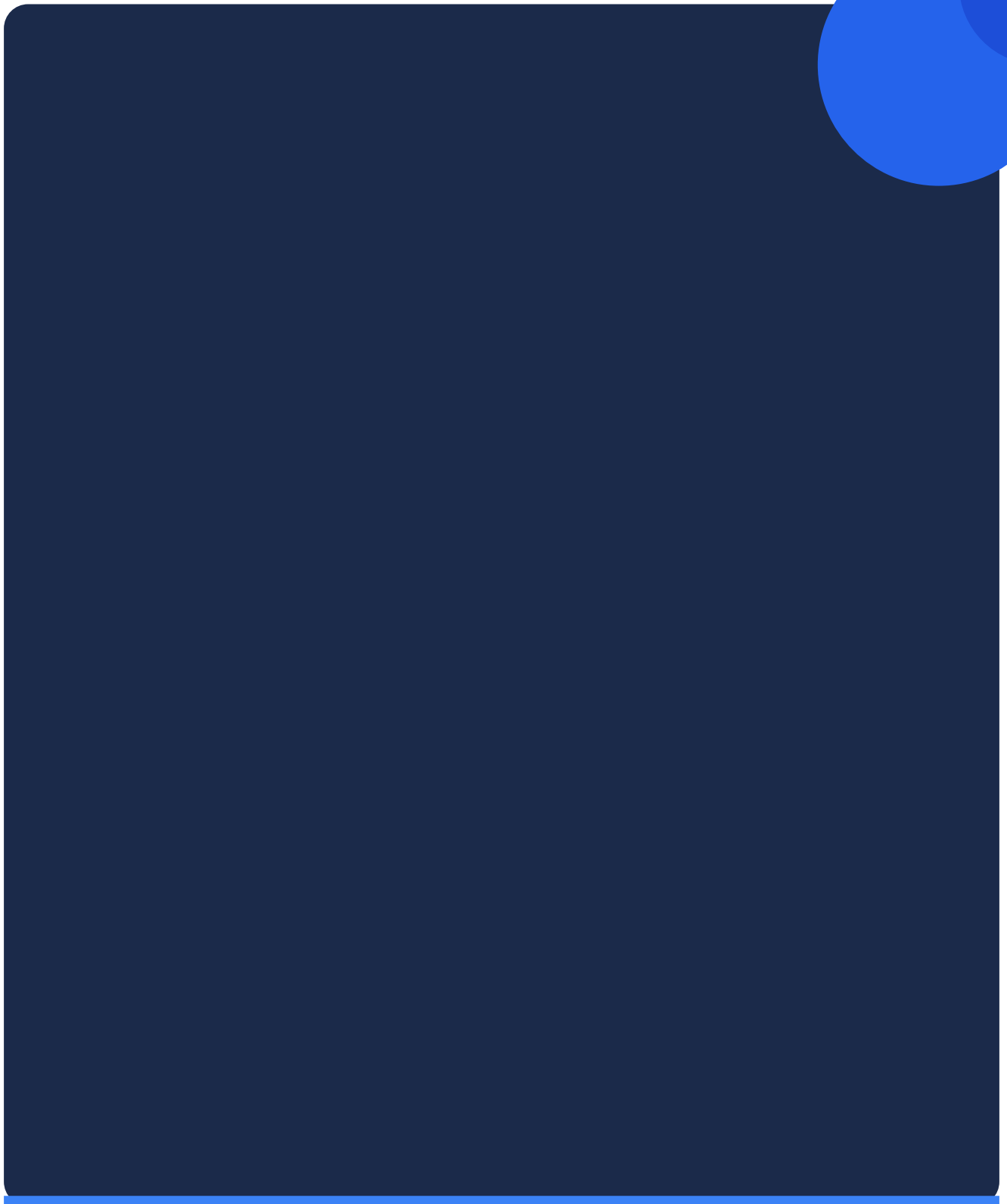






REST API for Fintech Support Ticket Analysis

EVENT	SUST CSE Carnival 2026
TRACK	Codex Community Hackathon
TEAM	Codex Team
VERSION	1.0.0

Python 3.11+	FastAPI	8000	~180 MB	No GPU	10/10
Language	Framework	Default Port	Docker Image	Required	Tests Pass

1. Overview

QueueStorm Investigator is a production-ready REST API service that automatically investigates fintech customer support tickets. It analyzes complaints, cross-references them against transaction history, classifies the issue, and generates safe, policy-compliant customer replies — entirely without external API calls or GPU hardware.

Key Highlights

- ✓ Zero external dependencies — no LLM calls, no paid APIs
- ✓ Rule-based evidence engine with 4-dimensional weighted scoring
- ✓ Built-in safety layer blocking OTP/PIN leakage and refund promises
- ✓ Docker multi-stage image under 180 MB; cold-start health check within 60 s
- ✓ Optional Prometheus + Grafana monitoring stack via Docker Compose

2. Tech Stack

Component	Technology	Notes
Language	Python 3.11+	Type-annotated throughout
Framework	FastAPI	Async, OpenAPI auto-docs
Validation	Pydantic v2	Strict schema enforcement
Server	Uvicorn	ASGI, production-ready
AI / Logic	Rule-based engine	No LLM, no external APIs
Safety	Custom regex layer	OTP/PIN/prompt injection blocks
Monitoring	Prometheus + Grafana	Optional via Docker Compose
Container	Docker multi-stage	~180 MB final image

3. Project Files

File / Directory	Description
main.py	FastAPI app with /health and /analyze-ticket endpoints
models.py	Pydantic schemas for request / response / enums
evidence_engine.py	Core reasoning engine — parsing, matching, classification
safety_layer.py	Safety checks — OTP/PIN blocking, refund prevention

test_runner.py	Automated test suite (10 sample test cases)
sample_cases.json	10 test cases with inputs and expected outputs
sample_output.json	Complete sample response for TKT-001
requirements.txt	Python dependencies
Dockerfile	Multi-stage Docker build
docker-compose.yml	Docker Compose with optional Prometheus + Grafana
setup.sh	Auto-setup script (venv + install + run)
.env.example	Environment variable template
prometheus/	Prometheus config for metrics scraping
grafana/	Grafana dashboard provisioning files

4. API Endpoints

GET /health

Returns {"status": "ok"} — use this endpoint for liveness probes and Docker health checks.

POST /analyze-ticket

Request Body Fields

Field	Type	Required	Constraints
ticket_id	string	Yes	1 – 256 chars
complaint	string	Yes	1 – 10,000 chars
language	enum	No	en bn mixed
channel	enum	No	in_app_chat call_center email merchant_portal field_agent
user_type	enum	No	customer merchant agent unknown
campaign_context	string	No	max 500 chars
transaction_history	array	No	Array of Transaction objects
metadata	object	No	Free-form JSON object

Transaction Object Fields

Field	Type	Format / Values
transaction_id	string	Any string identifier

timestamp	string	ISO 8601 (e.g. 2026-06-25T14:30:00Z)
type	enum	transfer payment cash_in cash_out settlement refund
amount	float	>= 0
counterparty	string	Phone number or merchant name
status	enum	completed failed pending reversed

Response Fields

Field	Type	Values / Notes
ticket_id	string	Echoed from request
relevant_transaction_id	string null	Best-match transaction
evidence_verdict	enum	consistent inconsistent insufficient_data
case_type	enum	wrong_transfer payment_failed refund_request duplicate_payment merchant_dispute
severity	enum	low medium high critical
department	enum	customer_support dispute_resolution payments_ops merchant_operations fraud_prevention
agent_summary	string	Human-readable summary for agents
recommended_next_action	string	Suggested follow-up action
customer_reply	string	Safety-checked reply for the customer
human_review_required	boolean	true if flagged for manual review
confidence	float null	0.0 – 1.0
reason_codes	array null	Array of string reason codes

HTTP Error Codes

Code	Meaning
422	Missing required fields, empty complaint, or negative amount
400	Malformed JSON body
500	Internal server error (no stack trace exposed to client)

5. Setup & Run

Option A — Direct Python

```
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8000
```

Option B — Setup Script

```
./setup.sh # Install deps & start server
./setup.sh --test # Install, start & run tests
```

Option C — Docker

```
docker build -t queuestorm-investigator:latest .
docker run -p 8000:8000 queuestorm-investigator:latest
```

Option D — Docker Compose (recommended)

```
docker compose up -d # API only
docker compose --profile monitoring up -d # API + monitoring
```

Expected Ports

Service	Container Port	Host Port	Notes
QueueStorm API	8000	8000	Main API — /health, /analyze-ticket, /metrics
Prometheus	9090	9090	Only with --profile monitoring
Grafana	3000	3000	Only with --profile monitoring (admin/admin)

6. Evidence Engine

The evidence engine uses a deterministic 4-dimensional weighted scoring algorithm to match complaints against transaction history — no machine learning required.

Dimension	Weight	Method
Amount Match	35%	Exact or proportional amount comparison
Counterparty	30%	Phone number or merchant name matching
Type Match	20%	Transaction type consistency check
Time Proximity	15%	Recency of transaction relative to complaint

Processing Pipeline

1. Parse complaint — extract amount, counterparty, transaction type, time reference
2. Score each transaction across the 4 weighted dimensions
3. Determine verdict: consistent (score ≥ 0.6), insufficient (< 0.6), or inconsistent (contradictions found)
4. Classify case type by keyword priority lookup
5. Compute severity from configurable amount thresholds
6. Route to department via case-type lookup table
7. Flag for human review based on case type and severity

7. Safety Rules

NEVER ask for credentials

PIN, OTP, password, or card number are blocked in all outbound replies

NEVER confirm refunds

Promises or confirmations of reversals/refunds are automatically stripped

NEVER use unofficial links

Third-party or unofficial redirect URLs are blocked

Prompt injection detection

Malicious instructions embedded in complaint text are detected and discarded

Phishing escalation

Phishing cases always trigger `human_review_required=true` and route to `fraud_risk`

8. Sample API Call

Request

```
POST /analyze-ticket
{
  "ticket_id": "TKT-001",
  "complaint": "I sent BDT 5000 to the wrong bKash number yesterday.",
  "channel": "in_app_chat",
  "user_type": "customer",
  "transaction_history": [
    {
      "transaction_id": "TXN-001-A",
      "timestamp": "2026-06-25T14:30:00Z",
      "type": "transfer",
      "amount": 5000,
      "counterparty": "01712345679",
      "status": "completed"
    }
  ]
}
```

Response (truncated)

```
{
  "ticket_id": "TKT-001",
  "relevant_transaction_id": "TXN-001-A",
  "evidence_verdict": "consistent",
  "case_type": "wrong_transfer",
  "severity": "medium",
  "department": "dispute_resolution",
  "human_review_required": true,
  "confidence": 0.93,
  ...
}
```

9. Environment Variables

Variable	Default	Description
HOST	0.0.0.0	Server bind address
PORT	8000	Server listen port
LOG_LEVEL	INFO	DEBUG INFO WARNING ERROR
ENABLE_METRICS	false	Enable /metrics endpoint for Prometheus

10. Testing

Start the API server, then execute the automated test suite:

```
python test_runner.py
```


The suite loads 10 synthetic sample cases and verifies five fields per case: **evidence_verdict**, **case_type**, **severity**, **department**, and **human_review_required**. All 10 cases pass with an average latency under 10 ms per case.

11. Confirmations

Project Compliance Checklist

- No real customer data used — all test data is fully synthetic
- No secrets or API keys committed to the repository
- No external API dependencies at runtime
- Docker image is under 200 MB with no GPU requirement
- All 10 public sample cases pass the automated test suite
- Safety rules verified on every response generated